Corrective RAG (CRAG) — Study Notes

Notes on the limitations of traditional RAG, the Corrective RAG (CRAG) architecture from the original paper, and a from-scratch, iterative implementation in LangGraph — covering knowledge refinement, retrieval evaluation, web-search fallback, query rewriting, and ambiguous-case handling.

1 Recap: How Traditional RAG Works

A traditional RAG (Retrieval-Augmented Generation) workflow has three steps:

1. **Retrieval** — the user's query is converted to a vector (via an embedding model) and used to perform a semantic search against a vector database, returning the closest-matching documents.
2. **Augmentation** — the retrieved documents are combined with the original query and sent to an LLM, instructing it to answer using those documents.
3. **Generation** — the LLM produces an answer based on the provided documents.

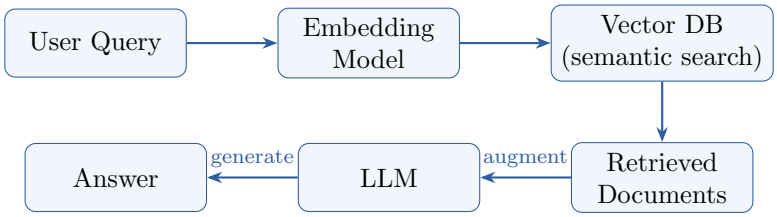

Figure 1: Traditional RAG: retrieval → augmentation → generation, with no check on document relevance.

2 The Problem: Blind Trust in Retrieved Documents

Traditional RAG **blindly trusts** whatever documents are retrieved. If the vector database has nothing genuinely relevant to the query, a semantic search will still return *something* (the closest match, however unrelated) — and the LLM is instructed to answer strictly from that context regardless of its actual relevance.

Example (demonstrated in code): A vector store containing only machine learning books was queried with "What is an LLM?" The retriever returned documents about random forests, MLPs, CNNs, and regularization — none discussing transformers. The LLM still produced a plausible-sounding answer about transformers, but purely from its own **parametric knowledge**, not from the (irrelevant) retrieved context. In other scenarios where the LLM lacks relevant parametric knowledge (e.g., a company's leave policy that doesn't exist in the vector database), this same blind trust can instead lead to outright **hallucination** — a serious risk in business-critical applications.

Corrective RAG (CRAG) exists specifically to address this failure mode.

3 Corrective RAG: The Core Idea

CRAG introduces a **retrieval evaluator** — a model that looks at the query and the retrieved documents and decides whether they are actually useful for answering the query, before generation happens. Based on this evaluation, one of three cases applies:

1. **Correct** — the retrieved documents are relevant. Proceed as in normal RAG: refine and generate from them.
2. **Incorrect** — the retrieved documents are not relevant at all. Instead of using them, CRAG goes to an **external knowledge source** (e.g., a web search) and generates the answer from that instead.
3. **Ambiguous** — the documents are partially relevant. CRAG uses *both*: the relevant retrieved documents plus supplementary results from a web search, merging them before generation.

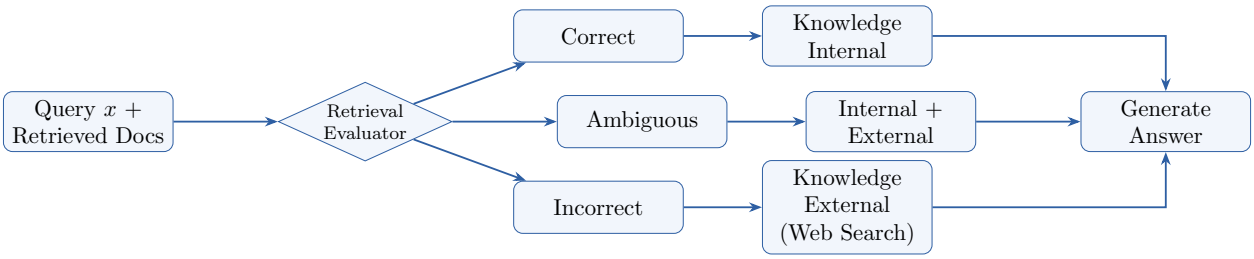

Figure 2: Core CRAG idea: the retrieval evaluator routes to one of three cases, each producing internal and/or external knowledge that feeds generation.

This design is based on the original CRAG research paper (2024), whose architecture diagram shows: query $x \rightarrow$ retrieved documents $(d_1, d_2) \rightarrow$ retrieval evaluator $\rightarrow$ depending on the verdict, refine into *Knowledge Internal*, fetch *Knowledge External* via web search, or both — which then feeds generation together with x.

4 Building CRAG Iteratively in LangGraph

Rather than building the full architecture at once, it is built up in stages, starting from traditional RAG and adding one feature at a time.

4.1 Baseline Traditional RAG Setup

- Three classic ML/DL books are loaded as documents, split via a recursive character text splitter (chunk size 900, overlap 150).
- Chunks are embedded (OpenAI embeddings) and stored in a FAISS vector database; a retriever returns the top 4 documents for a query.
- State holds: question, retrieved docs, and answer.
- Graph: `retrieve → generate`, where generation is instructed to answer only from context and say "I don't know" otherwise.

4.2 Iteration 1: Knowledge Refinement

Motivation: Retrieved documents (chunks) often mix relevant content with irrelevant material, since text-chunking (e.g., splitting every 900 characters) doesn't respect topic boundaries — a single chunk can span multiple topics, or one topic can be split across chunks.

Knowledge refinement has three steps, based on the paper:

1. **Decomposition** — break each retrieved document into "strips" (roughly sentence-level units).
2. **Filtration** — for each strip, a model scores whether it's relevant to the query. (The paper uses a fine-tuned T5-Large model for this; since that fine-tuned checkpoint isn't publicly available, an LLM — e.g., ChatOpenAI — is substituted instead, with structured output forcing a true/false "keep" decision.)
3. **Recomposition** — the strips marked relevant ("kept") are merged back together into a refined context, which is what's actually used for generation — instead of the full, noisier original document.

State additions: `strips` (all decomposed sentence-level pieces), `kept_strips` (those judged relevant), and `refined_context` (the recombined kept strips).

Implementation: A helper function splits text into sentence-level strips. A refine node: fetches the question and all retrieved docs, joins and decomposes them into strips, runs each strip through an LLM filter chain (structured output: keep = true/false), collects the kept strips, joins them into `refined_context`, and returns all three state fields. The graph becomes `retrieve → refine → generate`. Testing showed the refined, more targeted context still produced a correct — and slightly more focused — answer.

4.3 Iteration 2: Retrieval Evaluation (Thresholding Logic)

This adds the retrieval evaluator itself, which decides whether retrieval quality is good, bad, or in between.

An LLM is given the query and each retrieved document individually, and asked to output a relevance score between 0 and 1 (plus a short reason). Two thresholds are defined: a **lower threshold** (e.g., 0.3) and an **upper threshold** (e.g., 0.7).

- **Correct** — at least one document scores above the upper threshold.
- **Incorrect** — no document scores above the lower threshold (i.e., all documents score at or below it).
- **Ambiguous** — anything else (some signal, but no single document is strongly relevant).

Important detail: regardless of verdict, only documents scoring **above the lower threshold** ("good docs") are used for generation — a document with, say, a low score is excluded from generation even in a "correct" case if it individually falls below the lower threshold.

(As with refinement, the paper uses a fine-tuned T5-Large model here too, citing lower cost and better task-specific performance versus a general LLM; since that checkpoint isn't available, an LLM is substituted.)

Implementation: A Pydantic schema captures a score and a reason per document. A system prompt instructs the LLM to act as a "strict retrieval evaluator," returning a relevance score

(0 to 1, conservative with high scores) and a short reason for each document via structured output. New state fields: `good_docs` (docs scoring above the lower threshold), `verdict` (correct/incorrect/ambiguous), and `reason`. An evaluation node loops over all retrieved documents, scores each, populates `good_docs`, and determines the verdict per the thresholding rules above.

At this stage, only the "correct" path is fully wired to refinement + generation; "incorrect" and "ambiguous" verdicts simply print their status without generating an answer yet (those are added in later iterations).

4.4 Iteration 3: Web Search Integration (Incorrect Path)

When the verdict is "incorrect," instead of returning nothing useful to the user, the system performs a **web search** (via Tavily) using the original query, treats the search results as documents, and generates an answer from those instead.

> **Key detail from the paper:** web search results are not used directly either — they go through the *same* refinement process (decomposition → filtration → recomposition) as retrieved documents, since not all search results are equally useful. The refined web results become *Knowledge External*.

Implementation: A new state field `web_docs` stores documents obtained from Tavily search (title, URL, and content extracted per result, converted into document objects). The refine node is updated: if the verdict is "correct," build context from `good_docs`; if "incorrect," build context from `web_docs` instead — the rest of the refine/generate logic is reused unchanged. A new web-search node replaces what was previously a simple "fail" node for the incorrect path. Testing with a query outside the books' scope (e.g., recent AI news) showed the system correctly identifying "incorrect," performing a web search, and generating a grounded answer from the fetched results — rather than failing outright.

4.5 Iteration 4: Query Rewriting

> Before sending a query to a web search engine, the original user query is **rewritten** into a more search-engine-friendly form — since user queries can be vague, underspecified, or missing keywords/time constraints, while search engines perform better with clear, keyword-rich, well-specified queries.

Example: a vague query like "recent AI news" might be rewritten to something like "recent AI news last 30 days" — adding an explicit recency constraint the LLM infers is implied.

Implementation: A new `rewrite_query` node runs (via an LLM with a Pydantic schema producing a single rewritten query) before the Tavily search node, whenever the path goes to web search. A system prompt instructs the LLM to act as a "web search query composer," keep it short, add recency constraints if implied, and never answer the question itself — only output a rewritten query. This rewritten query — not the original — is then sent to Tavily; everything downstream (refine, generate) is unchanged.

4.6 Handling the Ambiguous Case

For the ambiguous verdict, both retrieved documents *and* web search results are used and merged:

- The already-good retrieved documents (`good_docs`) are kept.
- A web search is also performed (with query rewriting) to fetch `web_docs`.
- The refine node builds its context from **both** `good_docs` and `web_docs` combined, then applies the same decomposition/filtration/recomposition process to the merged set before generation.

This lets the routing logic simplify to just two branches: if the verdict is "correct," go straight to refine (using only `good_docs`); if the verdict is "incorrect" *or* "ambiguous," go through rewrite query → web search → refine (using `good_docs` + `web_docs` for ambiguous, or `web_docs` alone for incorrect). The separate "ambiguous" branch and node are eliminated entirely — state naturally handles the merge.

Testing with a query partially covered by the source material (e.g., comparing two concepts where only one is discussed in the books) correctly produced an "ambiguous" verdict, triggered a web search, and generated an answer using both the internal and external knowledge merged together.

5 Final Architecture Summary

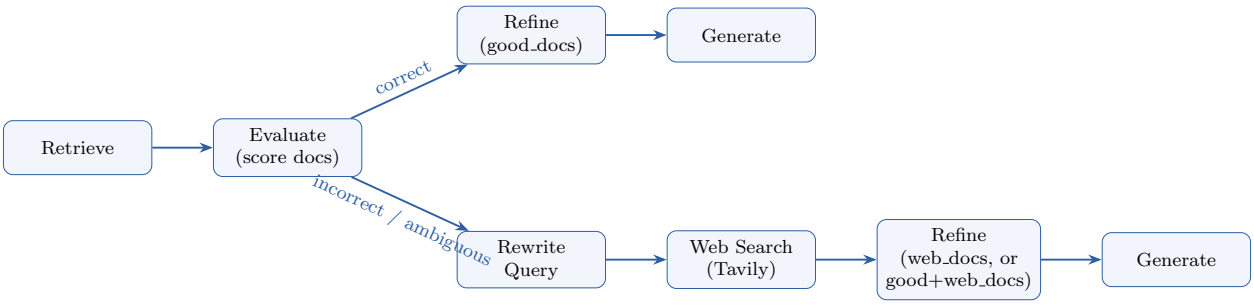

Figure 3: Final LangGraph CRAG architecture. The correct path refines only `good_docs`; the incorrect/ambiguous path shares a rewrite → web-search → refine → generate pipeline, differing only in which documents (web-only vs. good+web) feed the refine step.

The completed system mirrors the original CRAG paper's architecture:

- **Retrieve** → **Evaluate** (produces correct / incorrect / ambiguous, using thresholded per-document scores).
- **Correct** → refine (internal knowledge only) → generate.
- **Incorrect** → rewrite query → web search → refine (external knowledge only) → generate.
- **Ambiguous** → rewrite query → web search → refine (internal + external knowledge merged) → generate.

The main deviation from the original paper is the substitution of a general-purpose LLM (ChatOpenAI) in place of the paper's fine-tuned, lightweight T5-Large model for both the filtration and evaluation tasks, since that specific fine-tuned checkpoint was not publicly available.

6 Quick Revision Summary

1. Traditional RAG blindly trusts retrieved documents; if retrieval fails to find relevant content, the LLM either answers from irrelevant context or falls back on its own parametric knowledge — risking hallucination.

2. CRAG adds a **retrieval evaluator** that scores retrieved documents and classifies retrieval quality as correct, incorrect, or ambiguous.
3. **Knowledge refinement** (decomposition into strips → filtration → recomposition) removes irrelevant sentences from otherwise-relevant chunks, improving generation quality.
4. Thresholding: correct = at least one document above the upper threshold; incorrect = no document above the lower threshold; ambiguous = otherwise. Only documents above the lower threshold are ever used for generation.
5. Incorrect retrieval triggers a **web search** (e.g., via Tavily) instead of giving up; web results are refined the same way as retrieved documents before use.
6. **Query rewriting** improves web search results by converting vague user queries into clearer, keyword-rich, search-engine-optimized queries before searching.
7. The ambiguous case merges good retrieved documents with web search results into a single refined context, allowing the ambiguous and incorrect paths to share most of their logic (both route through rewrite → search → refine, differing only in which documents feed the refine step).
8. The original paper used a fine-tuned T5-Large model for filtration and evaluation (cheaper and reportedly better for this specific task); a general LLM is a practical substitute when that model isn't available.